

x86-64 Assembly Language Programming with Ubuntu-3

남궁명수

Chapter14. 다중 소스 파일(Multiple Source Files).....	3
[14-1] Extern 문	3
[14-2] 예제: 합계와 평균	3
[14-2-1] Assembly Main	4
[14-2-2] Function Source	5
[14-2-3] Assemble and Link	7
[14-3] 고급 언어와의 인터페이스	7
[14-3-1] 예제: C++ 메인 / 어셈블리 함수	8
[14-3-2] 컴파일, 어셈블, 링크	8
Chapter15. 스택 버퍼 오버플로우(Stack Buffer Overflow)	10
[15-1] 스택 버퍼 오버플로우 이해하기	11
[15-2] 삽입할 코드(Code to Inject).....	14
[15-3] 코드 삽입(Code Injection)	19
[15-4] 코드 삽입 보호 기법	22
[15-4-1] 데이터 스택 스매싱 보호기(Stack Canaries).....	23
[15-4-2] 데이터 실행 방지(Data Execution Prevention, DEP)	24
[15-4-3] 주소 공간 배치 난수화(Address Space Layout Randomization, ASLR) ..	25
[15-4-4] 세 가지 보호 기법의 비교	26
[15-4-5] 결론.....	26
Chapter16. 명령줄 인자(Command Line Arguments)	27
[16-1] 명령줄 인자 분석(Parsing Command Line Arguments)	28
[16-2] 고급 언어 예제.....	29
[16-2-1] 운영체제 수준에서의 구조(개념적 이해)	31
[16-3] 인자 개수와 인자 벡터 테이블	31
[16-4] 어셈블리 언어 예제	33
Chapter17. 입출력 버퍼링(Input/Output Buffering)	37
[17-1] 왜 버퍼를 사용하는가?	37
[17-2] 버퍼링 알고리즘	39
Chapter18. 부동소수점 명령어(Floating-Point Instructions)	42
[18-1] 부동소수점 값(Floating-Point Values).....	42
[18-2] 부동소수점 레지스터(Floating-Point Registers)	43
[18-3] 데이터 이동	43
[18-4] 정수 / 부동소수점 변환 명령어	45

[18-5] 부동소수점 산술 명령어	49
[18-5-1] 부동소수점 덧셈	49
[18-5-2] 부동소수점 뺄셈	50
[18-5-3] 부동소수점 곱셈	51
[18-5-4] 부동소수점 나눗셈	53
[18-5-5] 부동소수점 제곱근	54
[18-5-6] 전체 구조 정리	54
[18-6] 부동소수점 제어 명령어	55
[18-7] 부동소수점 호출 규약	60
[18-8] 예제: Sum and Average	61
[18-8] 예제: Absolute Value	65
Chapter19. 병렬 처리(Parallel Processing)	68
[19-1] 분산 컴퓨팅(Distributed Computing)	69
[19-2] 다중 처리(Multiprocessing)	70
[19-2-1] POSIX Threads(pThreads)	70
[19-2-2] 레이스 조건(Race Conditions)	72
Chapter20. 인터럽트(Interrupts)	76
[20-1] 다중 처리(Multiprocessing)	76
[20-1-1] 인터럽트 분류	77
[20-1-2] 인터럽트 발생 시점(Interrupt Timing)	77
[20-1-3] 인터럽트 분류	78
[20-2] 인터럽트 유형과 수준	79
[20-2-1] 인터럽트 유형	79
[20-2-2] 특권 수준(Privilege Levels)	79
[20-3] 인터럽트 처리 과정	80
[20-3-1] 인터럽트 서비스 루틴(ISR)	80
[20-3-2] 처리 단계	80
[20-4] 인터럽트 처리 요약	82

Chapter14. 다중 소스 파일(Multiple Source Files)

프로그램의 크기와 복잡성이 증가함에 따라 프로그램의 서로 다른 부분을 서로 다른 소스 파일에 저장하는 것이 일반적이다.

이렇게 하면 소스 파일이 지나치게 커지는 것을 방지할 수 있고 여러 프로그래머가 프로그램의 서로 다른 부분을 더 쉽게 작업할 수 있다.

서로 다른 프로그래머가 작성한 함수 호출이라 하더라도 Chapter12. 함수에서 설명한 표준 호출 규약 덕분에 올바르게 함께 동작한다.

이는 고급 언어와 인터페이스하는 경우에도 마찬가지이다.

이 장에서는 여러 소스 파일로 구성된 소스 코드를 어떻게 작성하고 사용하는 지 보여주기 위해 간단한 어셈블리 예제를 제시한다.

또한, C/C++ 소스 파일과 인터페이스하는 예제도 제공된다.

[14-1] Extern 문

한 소스 파일에서 함수를 호출했는데 그 함수 코드가 현재 소스 파일에 없으면 어셈블러는 오류를 발생시킨다.

현재 파일에 없는 변수를 접근하는 경우도 마찬가지이다.

함수 코드나 변수가 다른 파일에 있음을 어셈블러에 알리기 위해 extern 문을 사용한다. 구문은 다음과 같다.

extern <symbolName>

symbolName 은 다른 소스 파일에 있는 함수 이름 또는 변수 이름이다.

일반적으로 여러 파일에서 접근하는 전역 변수를 사용하는 것은 좋은 프로그래밍 방식이 아니며, 가급적으로 사용하지 않는 것이 좋다.

데이터는 일반적으로 함수 호출 시 인수로 전달된다.

이 교재의 예제는 전역 변수 없이 외부 함수만 사용하는 데 초점을 둔다.

[14-2] 예제: 합계와 평균

다음은 간단한 main 예제로, stats()라는 어셈블리 함수를 호출하여 부호 있는 정수 리스트의 정수 합과 정수 평균을 계산한다.

main 과 함수는 서로 다른 소스 파일에 있으며, 다중 소스 파일 사용 방법을 보여주기 위한 예제이다.

사실 이 예제는 여러 소스 파일이 필요할 만큼 크지 않다.

[14-2-1] Assembly Main

main 코드는 다음과 같다.

```
; Simple example to call an external function.
; -----
; Data section

section .data
; -----
; Define standard constants

LF equ 10
NULL equ 0
TRUE equ 1
FALSE equ 0
EXIT_SUCCESS equ 0
SYS_exit equ 60
; -----
; Declare the data

lst1 dd 1, -2, 3, -4, 5
dd 7, 9, 11
len1 dd 8

lst2 dd 2, -3, 4, -5, 6
dd -7, 10, 12, 14, 16
len2 dd 10

section .bss

sum1 resd 1
ave1 resd 1
sum2 resd 1
ave2 resd 1

; -----

extern stats

section .text
global _start
```

```

_start:
; ----
; Call the function
; HLL Call: stats(lst, len, &sum, &ave);

mov rdi, lst1
mov esi, dword [len1]
mov rdx, sum1
mov rcx, ave1
call stats

mov rdi, lst2
mov esi, dword [len2]
mov rdx, sum2
mov rcx, ave2
call stats
; -----
; Example program done

exampleDone:
mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall

```

[14-2-2] Function Source

다른 소스 파일에 있는 함수 코드는 다음과 같다.

```

; Simple example void function.
; *****

section .data
section .text
; -----
; Function to find integer sum and integer average
; for a passed list of signed integers.
;
; Call:
; stats(lst, len, &sum, &ave);
;
; Arguments Passed:
; 1) rdi - address of array

```

```

; 2) rsi – length of passed array
; 3) rdx – address of variable for sum
; 4) rcx – address of variable for average
;
; Returns:
; sum of integers (via reference)
; average of integers (via reference)

global stats
stats:

push r12

mov r11, 0
mov r12d, 0

sumLoop:
mov eax, dword [rdi+r11*4]
add r12d, eax
inc r11
cmp r11, rsi
jb sumLoop

mov dword [rdx], r12d
; -----
; Find and return average.

mov eax, r12d
cdq
idiv esi
mov dword [rcx], eax

; -----
; Done, return to calling function.

pop r12
ret

```

위 소스 파일은 Chapter 5. 툴체인에서 설명한 것과 동일한 assemble 명령으로 조립할 수 있다.

외부 함수를 호출하지 않으므로 extern 문은 필요 없다.

[14-2-3] Assemble and Link

main 소스 파일 이름이 main.asm 이고

함수 소스 파일 이름이 stats.asm 이라고 가정하면 다음 명령으로 어셈블 및 링크를 수행할 수 있다.

```
yasm -g dwarf2 -f elf64 main.asm -l main.lst
yasm -g dwarf2 -f elf64 stats.asm -l stats.lst
ld -g -o main main.o stats.o
```

파일 이름은 원하는 대로 변경할 수 있다.

디버거는 다음 명령으로 실행한다.

```
ddd <executable>
```

디버거의 step 명령은 함수 내부로 진입하며 다른 파일에 있는 함수 소스 코드도 보여준다.
next 명령은 함수 전체를 실행하고 함수 소스 코드는 표시하지 않는다.

함수가 정상 동작한다면 next 명령이 더 유용하다.

함수를 디버깅하려면 step 명령이 더 유용하다.

“-g” 옵션을 생략하면 디버거는 소스 코드를 표시할 수 없다.

[14-3] 고급 언어와의 인터페이스

이 절에서는 고급 언어가 어셈블리 언어 함수를 호출하는 방법과

어셈블리 언어 함수가 고급 언어 함수를 호출하는 방법에 대해 설명한다.

이 장에서는 두 경우 모두에 대한 예제를 제시한다.

간단히 말하면 이러한 작업은 표준 호출 규약을 통해 이루어진다.

따라서 고급 언어와 인터페이스할 때 추가적인 특별한 코드는 필요하지 않다.

다만, 컴파일러나 어셈블러는 외부 루틴에 대해 알려져야 한다.

그렇지 않으면, 현재 파일에 없는 루틴의 소스 코드를 찾을 수 없다는 오류 메시지가 발생한다.

여러 파일을 링크하는 일반적인 과정은 Chapter 5. 툴체인에서 설명되었다.

다중 소스 파일을 사용하는 과정은 Chapter 14. 다중 소스 파일에서 설명되었다.

객체 파일이 고급 언어에서 생성되었든 어셈블리 소스에서 생성되었든 상관 없다.

[14-3-1] 예제: C++ 메인 / 어셈블리 함수

별도의 소스 파일에 있는 함수를 호출할 때 컴파일러는 해당 함수의 소스 코드가 현재 소스 파일 외부에 있음을 알아야 한다.

이는 C 또는 C++에서 extern 문을 통해 수행된다.

다른 언어도 유사한 구문을 가진다.

고급 언어에서는 extern 문에 함수 프로토타입이 포함되며 이를 통해 컴파일러는 함수 인수와 관련 타입을 검증할 수 있다.

다음은 C++ main 과 어셈블리 함수 예제이다.

```
#include <iostream>
using namespace std;

extern "C" void stats(int[], int, int *, int *);

int main()
{
    int lst[] = {1, -2, 3, -4, 5, 7, 9, 11};
    int len = 8;
    int sum, ave;

    stats(lst, len, &sum, &ave);

    cout << "Stats:" << endl;
    cout << " Sum = " << sum << endl;
    cout << " Ave = " << ave << endl;

    return 0;
}
```

여기서 참조된 stats()함수는 이전 예제의 것을 수정 없이 그대로 사용해야 한다.

[14-3-2] 컴파일, 어셈블, 링크

함수 장에서 언급했듯이 C++ 컴파일러를 사용해야 한다.

예를 들어, C++ main 파일 이름이 main.cpp 이고 어셈블리 소스 파일 이름이 stats.asm 이라고 가정하면 컴파일, 어셈블, 링크 및 실행 명령은 다음과 같다.

```
g++ -g -Wall -c main.cpp
yasm -g dwarf2 -f elf64 stats.asm -l stats.lst
g++ -g -o main main.o stats.o
```


Ubuntu 18 이상에선 다음과 같이 `-no-pie` 옵션이 필요하다.

```
g++ -g -no-pie -o main main.o stats.o
```

파일 이름은 원하는 대로 변경할 수 있다.

실행 결과는 다음과 같다.

```
./main  
Stats:  
Sum = 30  
Ave = 3
```

C main 을 사용하는 경우, main 파일 이름이 `main.c` 이고

어셈블리 소스 파일 이름이 `stats.asm` 이라면 컴파일, 어셈블, 링크 및 실행 명령은 다음과 같다.

```
g++ -g -Wall -c main.cpp  
yasm -g dwarf2 -f elf64 stats.asm -l stats.lst  
g++ -g -o main main.o stats.o
```

Ubuntu 18 이상에서는 다음과 같이 `-no-pie` 옵션이 필요하다:

```
g++ -g -no-pie -o main main.o stats.o
```

링크 단계는 C 컴파일러 `gcc` 또는

C++ 컴파일러 `g++`를 사용한다.

이는 해당 컴파일러가 C/C++ 라이브러리의 적절한 위치를 알고 있기 때문이다.

Chapter15. 스택 버퍼 오버플로우(Stack Buffer Overflow)

스택 버퍼 오버플로우는 프로그램이 스택 기반 동적 변수를 초과하여 데이터를 저장할 때 발생한다.

여기서 말하는 스택 기반 동적 변수는
함수 내부에서 선언되는 로컬 변수를 의미한다.

[1] 기본 개념

예를 들어:

- 48 바이트 크기의 로컬 배열을 선언
- 그런데 사용자가 100 바이트를 입력
- 경계 검사 없이 저장

배열을 넘어선 메모리 영역이 덮어써진다. 이를 버퍼 오버플로우(Buffer Overflow)라 한다.

[2] 왜 스택에서 더 위험한가?

스택에는 단순한 변수만 있는 것이 아니다.

스택에는 다음과 같은 중요한 제어 정보가 포함되어 있다.

- 로컬 변수
- 보존된 레지스터(callee-saved)
- 프레임 포인터(rbp)
- 리턴 주소(rip)
- 스택 기반 매개변수

이 중 특히 리턴 주소(rip)가 손상되면 프로그램의 제어 흐름이 완전히 깨진다.

[3] 스택 스매싱(Stack Smashing)

버퍼 오버플로우를 의도적으로 악용하면 이를 스택 스매싱이라고 한다.

스택 구조는 표준 호출 규약 덕분에 매우 예측 가능하다.

즉 공격자는:

- 버퍼 위치
- 리턴 주소 위치
- 프레임 구조

를 계산할 수 있다.

이 특성 때문에 버퍼 오버플로우는 과거에 매우 강력한 공격 수단이었다.

[4] 이 장을 배우는 이유

이 장의 목적은 공격을 배우는 것이 아니라

- 취약점이 어떻게 발생하는지 이해
- 왜 위험한지 이해
- 어떻게 방어하는지 배우기를 위함

이다.

[5] 중요한 점

이 문제는 어셈블리만의 문제가 아니다.

C, C++같은 고급 언어에서도 발생한다.

어셈블리로 배우는 이유는 메모리 구조가 눈에 보이기 때문이다.

[15-1] 스택 버퍼 오버플로우 이해하기

[1] 함수 호출 시 스택 구조

다음 호출을 가정하자:

```
expFunc(arg1, arg2, arg3);
```

표준 호출 규약에 따라

1. 인수 전달
 2. 리턴 주소 push
 3. 프레임 포인터 저장
 4. 로컬 변수 공간 확보
- 이 과정을 거친다.

[2] 예제 함수 구조

expFunc 내부:

- 48 바이트 로컬 배열
- 8 바이트 로컬 변수(quadword)

총 56 바이트 로컬 데이터

[3] 스택 호출 프레임 구조

개념적으로 다음과 같다.

높은 주소

리턴 주소 (rip)

저장된 rbp

저장된 rbx

저장된 r12

저장된 r13

로컬 배열 (48 bytes)

로컬 quad 변수 (8 bytes)

낮은 주소

스택은 아래 방향으로 성장한다.

[4] 함수 종료 시

함수가 끝나면

1. 보존된 레지스터 복원
2. rbp 복원
3. ret 실행
4. 스택에서 리턴 주소 pop → rip 에 저장

호출자 함수로 복귀

[5] 오버플로우가 발생하면?

48 바이트 배열에 100 바이트를 쓴다고 가정하자.

메모리는 연속적이므로:

[배열] → [로컬 변수] → [저장된 레지스터] → [rbp] → [rip]

순서대로 덮어써진다.

최악의 경우:

- rbp 손상
- rip 손상

[6] rip 가 손상되면?

rip 는 다음 실행할 명령어 주소이다.

손상되면

- 함수가 호출 루틴으로 정상 복귀 불가
- 임의의 주소로 점프 시도
- 대부분 유효하지 않은 주소

Segmentation Fault 발생

[7] 왜 디버깅이 어려운가?

오류는 “리턴 시점”에 발생한다.

그러나 실제 문제는:

- 함수 내부에서
- 배열에 데이터를 저장할 때
- 경계 검사를 하지 않았기 때문

즉, 증상과 원인이 물리적으로 떨어져 있다.

[8] 고급 언어에서도 동일한 문제

C 함수 중 일부는 크기 검사를 하지 않는다.

대표적인 예:

```
strcpy(dest, src);
```

이 함수는 dest 의 크기를 전혀 확인하지 않는다.

따라서

- src 가 더 크면
- dest 를 초과하여 덮어쓴다.

이 때문에 strcpy 는 안전하지 않은 함수로 분류된다.

[9] 취약점 테스트 방법

간단한 테스트:

1. 프로그램이 이름 입력 요청
2. 200 자 입력
3. 프로그램 충돌 발생

취약점 존재

충돌하지 않고 오류 메시지 출력 → 보호 로직 존재

[10] 핵심 메커니즘 정리

스택 오버플로우의 본질은

데이터 영역 침범 → 제어 정보 파괴 → 제어 흐름 변조

특히:

버퍼 → rbp → rip

를 덮어쓰는 순간 프로그램 제어권이 공격자에게 넘어갈 수 있다.

[11] 왜 호출 규약이 공격을 가능하게 하는가?

표준 호출 규약은 다음을 고정한다.

- 리턴 주소 위치
- rbp 위치
- 레지스터 저장 위치
- 스택 정렬 방식

이 구조가 예측 가능하기 때문에 오버플로우 위치 계산이 가능하다.

[12] 현대 시스템의 방어 기법(개요)

현대 OS 는 다음과 같은 보호 기법을 사용한다.

- Stack Canary
- NX(Non-Executable Stack)
- ASLR(Address Space Layout Randomization)
- PIE(Position Independent Executable)

이 장에서는 이후에 이러한 방어 개념을 설명한다.

[15-2] 삽입할 코드(Code to Inject)

스택 버퍼 오버플로우가 실제로 악용되려면 단순히 리턴 주소를 깨뜨리는 것만으로는 충분하지 않다.

공격자는 다음 두 가지가 필요하다.

1. 실행될 코드(Payload)
2. 그 코드로 점프하도록 조작된 리턴 주소

이 절에서는 먼저 어떤 코드가 삽입되는지를 설명한다.

[1] 공격 환경 가정

프로그램은 다음과 같은 환경에서 실행된다고 가정한다.

- 서버 기반 프로그램
- 웹 또는 앱 프론트엔드 통해 접근
- 사용자에게 직접적인 콘솔 접근 없음

이상적으로는 사용자가 할 수 있는 일이 프로그램이 허용한 범위로 제한되어야 한다.

하지만 공격의 목표는 다음과 같다.

- 백엔드 서버에서 셸(shell)을 실행하여 콘솔 접근 권한을 획득하는 것

[2] exec 시스템 서비스

Linux 에는 다른 프로그램을 실행할 수 있는 시스템 호출이 있다.

x86-64 Linux 에서

execve → syscall 번호 59

만약 /bin/sh 를 실행하면 셸이 생성된다.

[2-1] 기본 exec 예제

```
mov rax, 59
mov rdi, progName
syscall
```

여기서

```
progName db "/bin/sh", NULL
```

레지스터 의미:

레지스터	의미
rax	시스템 호출 번호 (59)
rdi	실행할 프로그램 이름 주소

이 코드를 실행하면: 새로운 셸이 실행된다.

[3] 리스트 파일 이해

48C7C03B000000

이는

mov rax, 59

의 기계어 표현이다.

리스트 파일에는 다음이 포함된다.

1. 줄 번호
2. 코드 섹션 상대 주소
3. 기계어(16 진수)
4. 어셈블리 명령

[4] 문제점 1: NULL 바이트

기계어에 0x00 이 포함되어 있다.

예:

3B000000

여기에는 NULL 바이트가 포함된다.

문제:

- NULL 은 문자열 종료 문자
- 키보드로 직접 입력 불가
- 입력 함수가 중간에서 멈출 수 있음

따라서 NULL 을 제거해야 한다.

[5] 문제점 2: 절대 주소 사용

mov rdi, progName

는 데이터 섹션 주소를 사용한다.

하지만 공격자는:

- 삽입되는 위치를 정확히 모른다.
- 데이터 섹션을 사용할 수 없다.

따라서: 상대 주소 대신 스택을 사용해야 한다.

[6] 해결 방법

[6-1] rax 설정 방식 변경

기존:

mov rax, 59

- NULL 포함

대신:

```
xor rax, rax  
mov al, 59
```

이렇게 하면:

- rax 상위 56 비트는 xor 로 0
- al 에 59 만 설정
- NULL 바이트 없음

[6-2] 문자열을 스택에 배치

문자열:

```
"/bin/sh"
```

문제:

- 7 바이트
- NULL 필요
- 주소 참조 문제

해결:

```
"/bin/sh"
```

왜 슬래시가 두 개?

Linux 는:

```
/bin/sh
```

```
//bin/sh
```

을 동일하게 해석한다.

이렇게 하면 정확히 8 바이트가 된다.

[6-3] 리틀 엔디안 고려

x86-64 는 리틀 엔디안이다.

즉,

- 낮은 주소에 LSB
- push 시 값이 역순으로 보임

따라서:

```
0x68732f6e69622f2f
```

메모리에 저장되면

```
2f 2f 62 69 6e 2f 73 68
```

```
"/bin/sh"
```


[7] 최종 수정된 삽입 코드

```
xor rax, rax
push rax
mov rbx, 0x68732f6e69622f2f
push rbx
mov al, 59
mov rdi, rsp
syscall
```

[8] 동작 과정 분석

[8-1] `xor rax, rax`

`rax = 0`

[8-2] `push rax`

NULL 8 바이트를 스택에 저장

→ 문자열 종료 역할

[8-3] `mov rbx, “//bin/sh”`

문자열을 레지스터에 저장

[8-4] `push rbx`

스택에 문자열 저장

스택 상태:

```
rsp → “//bin/sh”
      NULL
```

[8-5] `mov al, 59`

`sys_execve` 호출 번호 설정

[8-6] `mov rdi, rsp`

`rsp` 는 스택의 `top` 를 가리키고 있음

`rdi` = 문자열 주소

[8-7] `syscall`

셸 실행

[9] 왜 `rsp` 를 사용하는가?

공격자는:

- 데이터 섹션 주소를 모름
- 코드 위치를 모름

하지만:

- rsp 는 항상 현재 위치를 정확히 가리킨다.

따라서:

mov rdi, rsp

는 항상 올바른 문자열 주소를 얻는다.

[10] NOP(0x90)의 역할

nop

기계어:

0x90

아무 동작도 하지 않는다.

공격에서는

- 코드 정렬
- NOP sled 생성
- 점프 오차 허용

등에 사용된다.

[11] 최종 삽입할 바이트

48 31 C0 50
48 BB 2F 2F 62 69 6E 2F 73 68
53
B0 3B
48 89 E7
0F 05
90 90

이 바이트들을 메모리에 넣으면 셸이 실행된다.

[15-3] 코드 삽입(Code Injection)

삽입할 코드가 준비되었고 실제로 입력할 수 있다면, 다음 단계는 그 코드를 실행시키는 것이다.

앞 절(15.2)에서 준비한 코드는 exec 시스템 호출을 이용하여 /bin/sh 를 실행하는 셸코드였다.

이 코드는 Linux x86-64 환경에서 시스템 호출 번호 59 를 사용하여 셸을 생성한다.

[1] 기본 개념: 실행 흐름을 탈취한다는 것

스택 버퍼 오버플로우의 핵심은 단순히 데이터를 덮어쓰는 것이 아니다.

목표는 함수의 반환 주소(return address, RIP)를 조작하여, CPU 의 실행 흐름을 공격자가 삽입한 코드로 이동시키는 것이다.

함수 호출 시 스택에는 다음과 같은 정보가 저장된다.

- 지역 변수
- 보존된 레지스터들
- 저장된 베이스 포인터(saved RBP)
- 반환 주소(saved RIP)

이 중 가장 중요한 것은 저장된 RIP 이다.

함수가 종료되면 CPU 는 스택에 저장된 RIP 값을 꺼내 그 주소로 점프한다.

따라서 이 값을 공격자가 원하는 주소로 바꾸면

CPU 는 정상적인 호출자에게 돌아가지 않고 공격자가 삽입한 코드로 이동하게 된다.

[2] RIP 의 위치를 찾는 과정

스택에서 RIP 가 저장된 정확한 위치는 이론적으로 계산할 수도 있지만, 실제 실습 환경에서는 시행착오를 통해 찾는 경우가 많다.

입력을 점진적으로 늘려가며 다음과 같은 현상을 관찰한다.

[2-1] RBP 만 손상된 경우

- 프로그램이 이상 동작을 보이지만 즉시 충돌하지는 않을 수 있다.
- 호출 스택이 깨져 예기치 않은 오류가 발생한다.

[2-2] RIP 의 일부가 손상된 경우

- 프로그램이 함수에서 복귀하지 못한다.
- 대개 즉시 segmentation fault 가 발생한다.
- 잘못된 주소로 점프하려다 실패한다.

특히 8 바이트 주소 중 첫 바이트가 바뀌는 순간 프로그램은 거의 즉시 충돌한다.

이 지점을 통해 “아, 이제 RIP 에 도달했구나”를 판단할 수 있다.

[3] 공격 입력을 점진적으로 확장하는 방법

공격 코드는 보통 다음과 같이 구성된다.

[NOP NOP NOP ... NOP]

[실제 셸코드]

[padding]

[덮어쓸 RIP 주소]

입력은 여러 번의 시도에 걸쳐
한 번에 1 바이트씩 확장하면서 실험한다.

이 방식은 단순하지만 매우 효과적이다.
필요한 것은 계산 능력보다도 인내심이다.

[4] 반환 주소에 무엇을 써야 하는가?

RIP 위치를 알아냈다면 그 자리에 들어갈 8 바이트 값은 무엇이어야 할까?

정답은:

- 사용자 입력이 저장된 스택 영역 중 삽입된 코드가 위치한 주소

하지만 여기서 문제가 있다.

스택 주소는 정확히 어디일까?

실행 환경에 따라 약간씩 달라질 수 있다. 정확한 셸코드 시작 주소를 모를 수도 있다.

이 문제를 해결하기 위한 기법이 바로 다음 절에 나온다.

[5] NOP 슬라이드(NOP Slide)

nop 명령은 아무 동작도 하지 않는다.

기계어로는 0x90 이다.

CPU 가 nop 를 실행하면 아무 작업 없이 다음 명령으로 이동한다.

이 특성을 이용하면 다음과 같은 구조를 만들 수 있다:

[NOP NOP NOP NOP NOP NOP ...]

↓

[실제 셸코드]

반환 주소를 NOP 영역의 어느 지점으로 점프시키더라도 CPU 는 그 위를 미끄러지듯
이동하여 결국 셸 코드에 도달하게 된다.

이것이 바로 NOP 슬라이드이다.

중요한 이유:

- 정확한 셸코드 시작 주소를 몰라도 된다.

- 주소 계산 오차를 허용한다.

- 공격 성공 확률이 크게 증가한다.

[6] 스택 구조 예시 설명

스택 프레임이 손상된 모습은 대략 다음과 같다.

[변경된 return address]

[손상된 saved RBP]

[trailing NOP 들]

[NOP NOP NOP NOP]

[셸코드]

[NOP padding]

[Local array (48 bytes)]

[Local variable (8 bytes)]

[Preserved registers]

여기서 핵심은

- 지역 배열이 클수록
- 더 긴 NOP 슬라이드를 만들 수 있고
- 공격 성공 가능성이 높아진다.

즉, 큰 버퍼는 더 위험하다.

[7] 공격의 전체 흐름 정리

1. 취약한 함수에 긴 입력을 전달한다.
2. 입력이 지역 배열을 넘어서 스택을 덮는다.
3. saved RBP 를 지나 saved RIP 까지 도달한다.
4. saved RIP 를 NOP 슬라이드 주소로 덮어쓴다.
5. 함수가 반환된다.
6. CPU 가 조작된 RIP 로 점프한다.
7. NOP 영역을 따라 이동한다.
8. 셸코드를 실행한다.
9. /bin/sh 가 실행되어 셸이 생성된다.

[8] 로컬 배열이 클수록 위험한 이유

로컬 배열이 더 크다면 더 긴 NOP 슬라이드를 위한 더 많은 공간을 확보할 수 있다.

이는 단순한 설명이 아니라

실제 보안 설계에서 매우 중요한 원칙이다.

큰 고정 크기 버퍼는:

- 공격 입력을 더 많이 담을 수 있고
- 더 넓은 슬라이드를 만들 수 있으며
- 주소 추측 실패 확률을 낮춘다.

따라서 안전한 코드는:

- 버퍼 크기를 제한하고
- 입력 길이를 검증하며
- 스택 보호 기법(Stack Canary 등)을 사용해야 한다.

[9] 결론

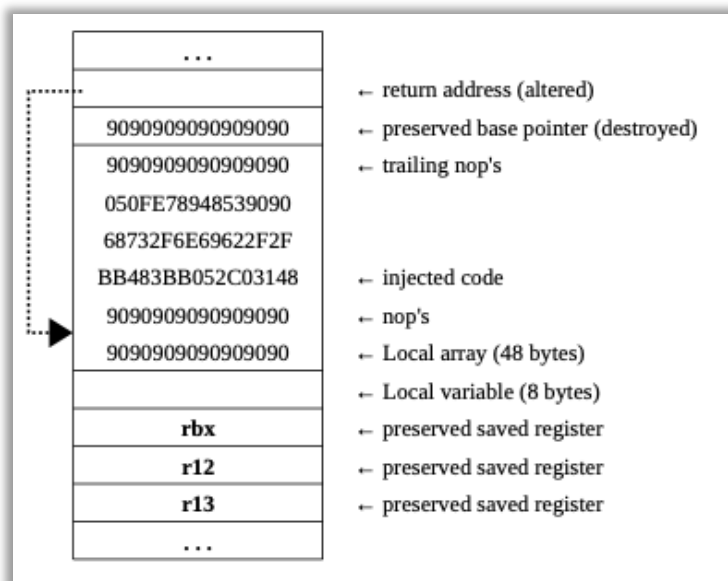
코드 삽입 공격의 핵심은 단순히 셸 코드를 준비하는 것이 아니라:

- 스택 구조를 이해하고
- RIP 위치를 찾아내고
- 반환 주소를 조작하며
- NOP 슬라이드를 이용해 정확도 문제를 해결하는 것

이 모든 고정은 결국 하나의 목표를 향한다.

프로그램의 제어 흐름을 탈취하여 공격자가 원하는 코드를 실행하는 것

이것이 바로 스택 버퍼 오버플로우 공격의 본질이다.



[15-4] 코드 삽입 보호 기법

스택 버퍼 오버플로우는 오랫동안 사용되어 온 공격 기법이며, 이에 대응하기 위해 다양한 보호 기술이 개발되어 왔다.

이 절에서는 대표적인 세 가지 보호 기법을 살펴본다.

1. 스택 카나리아(Stack Canaries)
2. 데이터 실행 방지(DEP)
3. 주소 공간 배치 난수화(ASLR)

중요한 점은:

이들 중 어느 것도 단독으로는 완벽하지 않다.

현대 시스템은 보통 여러 기법을 동시에 사용한다.

[15-4-1] 데이터 스택 스매싱 보호기(Stack Canaries)

[1] 기본 개념

스택 카나리아(Stack Canary)는 리턴 주소가 손상되었는지를 감지하기 위한 보호 기법이다.

이 방식은 다음과 같이 동작한다.

1. 함수 호출 시,
2. 리턴 주소(RIP) 바로 앞 위치에
3. 무작위 값(카나리아 값)을 삽입한다.

스택 구조는 다음과 같이 된다.

[Local Variables]
[Canary Value]
[Saved RBP]
[Return Address (RIP)]

버퍼 오버플로우가 발생하여 리턴 주소를 덮어쓰려면 그 전에 반드시 카나리아 값을 덮어써야 한다.

[2] 동작 원리

함수가 종료되기 직전, 컴파일러가 삽입한 코드가 다음을 수행한다.

- 스택에 저장된 카나리아 값이
- 원래 값과 동일한지 검사한다.

값이 변경되었다면:

- 스택이 손상되었다고 판단하고
- 즉시 프로그램을 종료한다.
- 일반적으로 `__stack_chk_fail()` 함수가 호출된다.

따라서 공격자는

- 리턴 주소를 정확히 조작하면서
- 동시에 카나리아 값을 정확히 복원해야 한다.

무작위 값이기 때문에 이는 매우 어렵다.

[3] 컴파일러 옵션

GNU 컴파일러(GNU Project)의 g++에서는

`-fstack-protector`

옵션이 기본 활성화되어 있다.

비활성화하려면:

`-fno-stack-protector`

를 사용한다.

이 장에서 설명하는 순수 스택 기반 셸코드 삽입 실습을 수행하려면 해당 보호 기능을 비활성화해야 한다.

[4] 한계점

- 카나리아 값을 유출할 수 있다면 우회 가능
- 버퍼가 아닌 다른 메모리 손상에는 무력할 수 있음
- 논리적 취약점은 막지 못함

즉, 카나리아는 탐지용 보호 기법이지
메모리 손상을 근본적으로 차단하는 기술은 아니다.

[15-4-2] 데이터 실행 방지(Data Execution Prevention, DEP)

DEP는 메모리 영역을 “실행 가능” 또는 “실행 불가”로 구분하는 하드웨어/OS 기반 보안 기법이다.

[1] 핵심 원리

- 코드 영역(Text section): 실행 가능
- 스택, 힙 등 데이터 영역: 실행 불가

스택에 삽입된 셸 코드는 데이터 영역에 존재한다.

DEP가 활성화된 시스템에서는 CPU가 해당 영역에 명령어를 실행하려 할 때 즉시 예외가 발생한다.

[2] 실제 구현

현대 x86-64 시스템에서는 NX bit(No-eXecute bit)를 통해 구현된다.

운영체제는 각 메모리 페이지에 대해 실행 권한을 설정한다.

예를 들어:

- 스택: RW(읽기/쓰기, 실행 불가)
- 코드: RX(읽기/실행 가능)

이 기능은 현대 Linux 시스템에서 기본적으로 활성화되어 있다.

[3] 효과

DEP 가 활성화되면:

- 스택에 셸 코드를 삽입해도 CPU 가 이를 실행할 수 없다.

따라서 전통적인 “스택 셸코드 삽입”방식은 실패한다.

[4] 한계

DEP 는 코드 삽입은 막지만 코드 재사용 공격은 막지 못한다.

예:

- 기존 실행 가능한 코드 영역을 재사용하는 기법
- Return-Oriented Programming(ROP)

즉, DEP 는 실행을 제한하지만 제어 흐름 조작 자체를 막지는 않는다.

[15-4-3] 주소 공간 배치 난수화(Address Space Layout Randomization, ASLR)

ASLR 은 공격자가 정확히 메모리 주소를 예측하지 못하도록 만드는 기법이다.

[1] 기본 개념

프로세스가 시작될 때 다음 영역들의 시작 주소를 무작위로 배치한다.

- 실행 파일의 기준 주소
- 스택
- 힙
- 공유 라이브러리

이로 인해 같은 프로그램이라도 실행할 때마다 메모리 배치가 달라진다.

[2] 필요한 이유

스택 오버플로우 공격에서 가장 중요한 점은

- 리턴 주소를 정확한 셸코드 위치로 설정하는 것

하지만 ASLR 이 활성화되어 있으면:

- 스택 주소가 매번 바뀐다.
- 라이브러리 함수 주소도 바뀐다.
- 공격자는 정확한 점프 위치를 알 수 없다.

이로 인해 공격의 성공 확률이 급격히 낮아진다.

[3] 한계

ASLR 도 완벽하지 않다.

- 주소 정보 유출 취약점이 있으면 무력화 가능
- 32 비트 환경에서는 랜덤화 범위가 제한적
- 반복 시도로 우회 가능할 수 있음

[15-4-4] 세 가지 보호 기법의 비교

보호 기법	무엇을 막는가	방식	한계
Stack Canary	리턴 주소 손상	무작위 값 검사	값 유출 시 우회 가능
DEP	데이터 영역 실행	실행 권한 제한	코드 재사용 공격 가능
ASLR	정확한 주소 예측	주소 무작위화	정보 유출 시 무력화

[15-4-5] 결론

스택 버퍼 오버플로우에 대한 현대적 방어는 단일 기술이 아니라 다층 방어 전략이다.

- 카나리아는 손상을 탐지하고
- DEP는 데이터 실행을 차단하며
- ASLR은 주소 예측을 어렵게 만든다.

이 세 가지가 함께 사용될 때 전통적인 코드 삽입 공격은 매우 어려워진다.

그러나 중요한 점은 보안은 절대적인 것이 아니라 공격 비용을 높이는 과정이다.

Chapter16. 명령줄 인자(Command Line Arguments)

이 장에서는 운영체제가 명령줄 인자를 어떻게 처리하는지, 그리고 어셈블리 언어 루틴이 그 인자에 어떻게 접근할 수 있는지를 요약한다.

명령줄 인자란 프로그램 이름 뒤에 명령줄에서 입력되는 문자열들을 의미한다.

즉,

```
./expProg one 42 three
```

여기서 one, 42, three 가 명령줄 인자이다.

[1] 명령줄 인자의 목적

명령줄 인자는 프로그램 실행 시 필요한 정보를 사전 입력 방식으로 전달하는 수단이다.

즉:

- 대화형 입력(prompt, scanf 등)을 사용하지 않고
- 실행 순간에 필요한 매개변수를 함께 전달할 수 있다.

예:

- 컴파일러에서 입력 파일 지정
- 출력 파일 지정
- 옵션 설정(-o, -Wall 등)
- 수치 계산 프로그램의 초기 값 설정

단점은:

매개변수가 잘못되면 전체 명령줄을 다시 입력해야 한다는 점이다.

[2] 명령줄 인자의 본질

명령줄 인자는 프로그램에 전달되는 문자열 집합이다.

중요한점은:

모든 인자는 문자열(string) 형태로 전달된다.

숫자처럼 보이는 값도 운영체제는 이를 단순한 문자 배열로 취급한다.

예:

```
42
```

는 정수 42가 아니라 문자 '4'와 '2'로 구성된 문자열이다.

따라서 숫자로 사용하려면 프로그램이 직접 변환해야 한다.

[16-1] 명령줄 인자 분석(Parsing Command Line Arguments)

[1] 운영체제의 역할

운영체제는 다음 작업을 수행한다.

1. 사용자가 입력한 명령줄을 읽는다.
2. 공백을 기준으로 문자열을 분리한다.
3. 각 문자열을 인자로 구성한다.
4. 그 정보를 프로그램에 전달한다.

운영체제는 단순히 “전달”만 할 뿐, 어떤 인자가 올바른지 판단하지 않는다.
유효성 검사는 프로그램의 책임이다.

[2] 인자 분리 규칙

운영체제는:

- 공백이 아닌 문자들의 연속을 하나의 인자로 간주한다.
- 인자 사이의 공백은 제거된다.
- 각 인자 사이에는 최소 하나의 공백이 필요하다.

예:

```
./expProg one 42 three
```

는 다음 네 개의 인자로 구성된다.

1. ./expProg
2. one
3. 42
4. three

중요한 점은

프로그램 이름 자체도 첫 번째 인자이다.

따라서 인자가 하나도 없더라도 최소 1 개의 인자는 항상 존재한다.

[3] 호출 규약 관점에서의 이해

명령줄 인자는 결국 프로그램에 전달되는 매개변수(parameter)이다.

즉, 프로그램은 운영체제에 의해 호출되는 함수라고 볼 수 있다.

따라서 일반적인 함수 호출 규약이 적용된다.

Linux x86-64 환경에서는 System V AMD64 ABI 호출 규약에 따라 매개변수가 달라진다.

고급 언어에서는 이 과정을 숨기지만, 어셈블리에서는 직접 다루어야 한다.

[16-2] 고급 언어 예제

C/C++에서는 명령줄 인자를 main 함수의 매개변수로 전달한다.

일반적인 형태:

```
int main(int argc, char * argv[])
```

또는

```
int main(int argc, char ** argv)
```

[1] argc

argc(argument count)

- 전달된 인자의 개수
- 프로그램 이름도 포함
- 항상 최소 1 이상

예:

```
./argsExp one 34 three
```

이면

```
argc = 4
```

[2] argv

argv(argument vector)

- 문자열 포인터 배열
- 각 인자의 시작 주소를 저장
- argv[0]은 프로그램 이름

구조적으로 보면:

```
argv
↓
+-----+
| ptr ----- | → "./argsExp"
+-----+
| ptr ----- | → "one"
+-----+
| ptr ----- | → "34"
+-----+
| ptr ----- | → "three"
+-----+
```

[3] 예제 프로그램 분석

다음 프로그램은 인자를 출력한다.

```
int main(int argc, char * argv[])
```

핵심 동작:

- argc 출력
- argv[0] 출력
- argv[1]부터 반복 출력

실행:

```
./argsExp one 34 three
```

출력:

```
Total arguments provided: 4
The name used to start the program: ./argsExp
The arguments are:
1: one
2: 34
3: three
```

[4] 문자열과 숫자 변환

매개변수 “34”는 문자열이다.

이를 정수로 사용하려면 변환이 필요하다.

예:

```
int value = stoi(argv[2]);
```

또는

```
int value = atoi(argv[2]);
```

하지만 다음을 반드시 고려해야 한다.

- 변환 실패 가능성
- 숫자가 아닌 입력
- 범위 초과

즉, 오류 검사가 필수이다.

[16-2-1] 운영체제 수준에서의 구조(개념적 이해)

프로그램 시작 시 스택에는 다음이 저장된다.

```
argc
argv[0] 주소
argv[1] 주소
...
NULL
실제 문자열들
```

따라서 어셈블리 언어에서는 이 구조를 직접 탐색하여 인자에 접근할 수 있다.
이 점이 이 장의 핵심 연결 고리이다.

[1] 전체 정리

개념	설명
명령줄 인자	프로그램 실행 시 전달되는 문자열
구분 방식	공백 기준
첫 번째 인자	프로그램 이름
전달 형태	문자열 배열
숫자 사용	프로그램이 직접 변환 필요
운영체제 역할	파싱 및 전달
프로그램 역할	해석 및 검증

[16-3] 인자 개수와 인자 벡터 테이블

운영체제는 메인 프로그램을 함수처럼 호출한다.
따라서 표준 호출 규약이 적용된다.

Linux x86-64 환경에선 System V AMD64 ABI 호출 규약이 사용된다.

매개변수	전달 레지스터
첫 번째 인자	rdi
두 번째 인자	rsi

따라서 프로그램 시작 시:

- rdi: argc(argument count)
- rsi: argv(argument vector 의 시작 주소)

[1] rdi: Argument Count(argc)

rdi 에는 정수형 인자 개수가 저장된다.

중요한 점:

- 프로그램 이름도 포함된다.
- 최소값은 항상 1 이다.

예:

```
./expProg one 42 three
```

이면:

```
argc = 4
```

[2] rsi: argument Vector(argv)

rsi 는 인자 벡터 테이블의 시작 주소를 가리킨다.

이 테이블은 다음과 같은 구조를 가진다.

- 각 인자 문자열의 시작 주소를 담은
- 8 바이트(워드) 포인터들의 배열

즉, argv 는 “문자열 배열”이 아니라 “문자열 주소 배열”이다.

[3] Argument Vector Table 의 메모리 구조

인자가 총 4 개라면 구조는 다음과 같다.

```
rsi —► [ argv[0] 주소 ] ; 프로그램 이름  
        [ argv[1] 주소 ] ; 첫 번째 인자  
        [ argv[2] 주소 ]  
        [ argv[3] 주소 ]
```

각 항목은 8 바이트 주소이다.

각 주소는 다시 다음과 같은 문자열을 가리킨다.

```
"/expProg" NULL  
"one" NULL  
"42" NULL  
"three" NULL
```

중요한 특징:

- 모든 문자열은 NULL(0)로 종료된다.
- 개행 문자는 포함되지 않는다.
- 운영체제가 문자열 저장을 담당한다.

[4] 메모리 관점에서의 이해

스택 상단에는 대략 다음 순서로 배치된다.

```
argc  
argv[0] 주소  
argv[1] 주소  
...  
NULL  
문자열 데이터들
```

따라서 어셈블리에서는:

```
[rsi + 0*8] → argv[0]  
[rsi + 1*8] → argv[1]  
[rsi + 2*8] → argv[2]
```

와 같이 접근할 수 있다.

[16-4] 어셈블리 언어 예제

이 예제는 명령줄 인자를 읽어 단순히 화면에 출력하는 프로그램이다.

핵심 개념은:

rdi 와 rsi 를 직접 사용하여 인자 테이블을 탐색하는 것

[1] main 진입 시 레지스터 상태

```
; rdi = argc  
; rsi = argv
```

코드는 이를 보존한다.

```
mov r12, rdi      ; argc 저장  
mov r13, rsi      ; argv 시작 주소 저장
```

이유:

- 이후 루프에서 사용할 것이기 때문
- 호출 규약에 따라 일부 레지스터는 함수 호출 시 변경될 수 있음

[2] 인자 출력 루프 분석

핵심 코드:

```
mov rbx, 0
```

rbx 는 인덱스 역할을 한다.

루프:

```
mov rdi, qword [r13+rbx*8]
call printString
```

동작 설명

- r13 = argv 시작 주소
- rbx*8 = 8 바이트 단위 인덱스
- qword [r13*rbx*8] = argv[n] 주소

그 주소를 rdi 에 넣고 printString 호출

즉,

```
argv[rbx]
```

를 출력한다는 의미이다.

[3] 루프 종료 조건

```
inc rbx
cmp rbx, r12
jl printLoop
```

r12 에는 argc 가 저장되어 있다.

따라서

```
rbx < argc
```

동안 반복한다.

결과적으로

```
argv[0]부터 argv[argc-1]까지
```

모두 출력된다.

[4] printString 함수 분석

이 함수는 NULL 종료 문자열을 출력하는 일반 루틴이다.

[4-1] 문자열 길이 계산

```
cmp byte [rbx], NULL
```

NULL 문자를 만날 때까지:

- rdx 에 길이 증가
- rbx 이동

[4-2] 시스템 호출

```
mov rax, SYS_write
mov rsi, rdi
mov edi, STDOUT
syscall
```

Linux 에서 write 시스템 호출은:

레지스터	의미
rax	시스템 호출 번호
rdi	파일 디스크립터
rsi	버퍼 주소
rdx	길이

이 코드는:

- STDOUT(1)에
- rdi 가 가리키는 문자열을 rdx 길이만큼 출력한다.

[5] 프로그램 종료

```
mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall
```

Linux 시스템 호출 번호 60 은 exit 이다.

[6] 빌드 과정의 의미

어셈블:

```
yasm -g dwarf2 -f elf64 cmdLine.asm -l cmdLine.lst
```

링크:

```
gcc -g -o cmdLine cmdLine.o
```

Ubuntu18 에서

```
gcc -g -no-pie -o cmdLine cmdLine.o
```

왜 -no-pie 가 필요한가?

현대 Linux 에서는 기본적으로 PIE(Position Independent Executable)가 활성화된다.

PIE 는 실행 파일의 주소를 무작위하여 ASLR 과 함께 동작한다.

그러나 이 예제는

- 고정된 진입점
- 표준 방식의 main 호출
- 단순한 링킹 환경

을 가정하므로 PIE 를 비활성해야 한다.

표준 링커를 직접 사용할 경우 C 런타임이 연결되지 않아
main 으로의 진입 방식이 달라질 수 있다.

Chapter17. 입출력 버퍼링(Input/Output Buffering)

이 장에선느 입출력(I/O) 버퍼링에 대해 설명한다.

I/O 버퍼링은 입출력 작업의 전체 처리량(throughput)과 효율을 향상시키기 위한 기법이다.

I/O 버퍼링이라는 주제는 매우 광범위하며 다음을 포함할 수 있다.

- 하드웨어 수준 버퍼
- 운영체제 수준 버퍼
- 네트워크 패킷 버퍼
- 응용 프로그램 패킷 버퍼

이 교재에서는 그 중에서도 파일로부터 정보를 읽기 위한 단일 버퍼 방식(single buffer buffering)을 다룬다.

비록 구체적인 응용이지만, 버퍼링의 기본 개념은 다른 모든 I/O 상황에 적용된다.

고급 언어(C/C++ 등)는 버퍼링을 수행하는 라이브러리 함수를 제공하여 관련 복잡성을 프로그래머로부터 숨긴다.

그러나 저수준 프로그래밍에서는 버퍼를 직접 설계하고 관리해야 한다.

이장의 목표는:

버퍼링이 왜 필요한지, 그리고 어떻게 동작하는지를 명확하게 이해하는 것이다.

[17-1] 왜 버퍼를 사용하는가?

버퍼링을 이해하려면 메모리 계층 구조를 떠올려야 한다.

메모리 접근 속도 비교

저장 장치	상대적 속도
CPU 레지스터	매우 빠름
주 메모리(RAM)	빠름
보조 저장장치(SSD/HDD)	매우 느림

보조 저장장치 접근은 주 메모리 접근보다 수십~수천 배 이상 느릴 수 있다.

[1] 시스템 호출의 오버헤드

파일 읽기 시스템 호출이 수행되는 과정:

1. 프로그램 실행 중 read() 호출
2. 제어가 운영체제(OS)로 이동
3. OS가 요청 검증
4. 디스크 컨트롤러에 요청 전달
5. 디스크에서 데이터 검색
6. Direct Memory Access(DMA)로 RAM에 복사
7. 완료 인터럽트 발생
8. OS가 프로그램 재개

이 전체 과정은 프로그램이 기다리는 동안 발생하는 시스템 오버헤드이다.

즉, 시스템 호출은 매우 비싼 작업이다.

[2] 한 문자씩 읽는 문제

저수준 read()는 다음을 요구한다.

몇 바이트를 읽을 것인가?

하지만 우리가 원하는 동작은 보통:

- 한 줄 읽기
- LF 까지 읽기

문제는: 줄 길이를 사전에 모른다.

컴퓨터 관점에서 파일은:

- 줄들의 집합이 아니라 연속된 바이트들의 집합이다.

LF 는 단지 ASCII 코드 10 일 뿐이다.

[3] 단일 문자 I/O 의 문제점

대화형 입력에서는:

- 사용자가 타이핑 속도를 제한
- 한 문자씩 읽어도 큰 문제 없음

하지만 파일 입력에서는:

- 대량 데이터
- 매우 빠른 공급 속도
- 단일 문자 read()는 지나치게 비효율적

예:

100,000 자를 한 문자씩 읽으면 100,000 번의 시스템 호출이 발생한다.

이는 성능적으로 매우 비효율적이다.

[4] 해결책: 버퍼 사용

대신 다음과 같이 한다.

1. 한 번의 시스템 호출로 큰 블록(예: 100,000 바이트)을 읽는다.
2. 주 메모리에 저장한다.
3. 이후 줄 단위 요청은 메모리에서 처리한다.
4. 버퍼가 소진되면 다시 파일에서 읽는다.

결과:

- 시스템 호출 횟수 감소
- 디스크 접근 감소
- 전체 성능 향상

[17-2] 버퍼링 알고리즘

이제 구체적인 알고리즘을 살펴본다.

가정:

- 파일은 이미 열려있음
- 우리는 myGetLine() 함수를 구현하려 한다.

```
status = myGetLine(fileDescriptor, textLine, MAXLEN);
```

[1] 파일 끝(EOF)의 판단

파일 끝은 특별한 코드가 존재하지 않는다.

대신:

요청한 바이트 수보다 적게 읽혔을 때 EOF로 판단한다.

예:

- 4096 바이트 요청
- 1200 바이트만 읽힘
- 파일 끝 도달

하지만 이미 버퍼에 읽어온 문자는 모두 처리해야 한다.

[2] 필요한 정적 변수

연속 호출 간 상태를 유지해야 하므로 정적(static) 변수가 필요하다.

변수 목록

변수	역할
BUFFER_SIZE	버퍼 크기
currIndex	현재 버퍼 위치
buffMaximum	현재 버퍼 내 유효 문자 수
eofFlag	파일 끝 여부

초기값:

```
currIndex = BUFFER_SIZE ; 첫 호출 시 강제로 읽기 수행  
buffMaximum = 0  
eofFlag = FALSE
```

[3] 알고리즘 흐름 해설

[3-1] 반복 구조

```
repeat {  
    if currIndex >= buffMaximum  
        read new buffer
```

- 버퍼가 비었으면
- 파일에서 새 블록 읽기

[3-2] 읽기 후 처리

```
if error → FALSE 반환  
if chars read < requested → eofFlag = TRUE  
reset currIndex
```

[3-3] 문자 처리

```
get one character from buffer  
store in textLine  
increment currIndex
```

[3-4] 줄 끝 검사

```
if character == LF  
    return TRUE
```

- LF를 만날 때까지
- 버퍼에서 문자 복사

[4] 중요한 설계 포인트

[4-1] 호출자는 버퍼 존재를 몰아야 한다.

myGetLine() 목표는: C++의 getline()처럼 동작하는 함수 제공
즉,

- 호출자는 버퍼 관리 세부사항을 몰라도 된다.
- 줄이 있으면 TRUE 반환
- 없으면 FALSE 반환

[4-2] 정적 변수 필요성

스택 변수는 함수 종료 시 사라진다.

하지만 우리는:

- 이전 호출에서 어디까지 읽었는지
- EOF 인지 여부

를 유지해야 한다.

따라서 정적 변수 사용이 필수이다.

[5] 처리하지 않는 문제들

원문 알고리즘은 다음을 완전하게 처리하지 않는다.

- 줄 길이가 maxLength 를 초과하는 경우
- 파일 크기가 정확히 버퍼의 배수인 경우
- 매우 긴 한 줄 처리
- 오류 복구

이는 실제 구현 시 반드시 보완해야 한다.

[6] 버퍼링의 핵심 개념 정리

개념	의미
버퍼	메모리 내 임시 저장 공간
목적	시스템 호출 감소
효과	성능 향상
문제	구현 복잡성 증가
해결	정적 상태 관리

Chapter18. 부동소수점 명령어(Floating-Point Instructions)

이 장에서는 x86-64 아키텍처에서 사용하는 부동소수점 명령어에 대한 기본 개념을 다룬다. 여기서는 가장 핵심이 되는 명령어들만 설명하며, 전체 목록은 부록에 정리되어 있다.

과거 32 비트 환경에서는 x87 FPU 스택 기반 명령어를 사용했지만, 이 교재에서는 x86-64 환경에서 사용하는 XMM 레지스터 기반 부동소수점 연산(SSE 계열)에 초점을 둔다.

명령어는 다음 순서로 설명된다.

- 데이터 이동
- 정수 ↔ 부동소수점 변환 명령어
- 산술 명령어
- 부동소수점 제어 명령어

중요한 특징은 다음과 같다.

- 부동소수점 연산은 특정한 하나의 고정 레지스터만 사용하는 구조가 아니다.
- 연산 과정에서 타입(32 비트/64 비트)이 자동으로 바뀌지 않는다.
- 즉, 우리가 어떤 타입을 사용할지 명확하게 지정해야 한다.

이러한 점은 오히려 부동소수점 명령어를 더 예측 가능하고 직관적으로 만들어 준다.

[18-1] 부동소수점 값(Floating-Point Values)

부동소수점 값은 일반적으로 다음 두 가지 정밀도로 표현된다.

- 단정밀도(Single Precision, 32 비트)
- 배정밀도(Double Precision, 64 비트)

C/C++에서의 대응 관계는 다음과 같다.

타입	크기	C/C++ 선언
단정밀도	32비트	float
배정밀도	64비트	double

어셈블리 명령어에서는 이를 구분하기 위해 다음과 같은 한정자(suffix)를 사용한다.

- s → single precision (32 비트)
- d → double precision (64 비트)

예:

- movss → scalar single(32 비트)
- movsd → scalar double(64 비트)

여기서 scalar 는 벡터가 아니라 하나의 값만 처리한다는 의미이다.

[18-2] 부동소수점 레지스터(Floating-Point Registers)

부동소수점 연산을 위해 XMM 레지스터라는 전용 레지스터 집합이 존재한다.

- 이름: xmm0~xmm15
- 총 16 개
- 기본 크기: 128 비트
- 최신 프로세서에서는 확장되어 256 비트(AVX)도 존재

그러나 이 교재에서는 가장 기본적인 형태로:

- 하위 32 비트(단정밀도)
- 상위 64 비트(배정밀도)

만을 사용한다.

중요한 점:

부동소수점 연산은 반드시 XMM 레지스터를 사용해야 한다.

일반 목적 레지스터(RAX, RBX 등)에서는 부동소수점 연산을 직접 수행할 수 없다.

즉, 계산을 하려면:

1. 메모리 → XMM 레지스터로 이동
2. 연산 수행
3. 결과 → 메모리 저장

이라는 단계를 거쳐야 한다.

[18-3] 데이터 이동

연산을 수행하려면 먼저 데이터를 XMM 레지스터로 로드(load)해야 한다.

계산이 끝나면 결과를 다시 메모리에 저장(store)할 수 있다.

이를 위해 사용하는 기본 명령어가 movss 와 movsd 이다.

[1] movss

movss <dest>, <src>

- 32 비트 단정밀도 부동소수점 값으로 이동
- 소스의 32 비트를 목적지의 32 비트로 복사
- 소스는 변경되지 않음

제약 조건

- 두 피연산자가 모두 메모리일 수 없다.
- 즉시값은 사용할 수 없다.
- 한쪽은 메모리, 한쪽은 레지스터

예시

```
movss xmm0, dword [x]
movss dword [fltSVar], xmm1
movss xmm3, xmm2
```

[2] movsd

movsd <dest>, <src>

- 64 비트 배정밀도 부동소수점 값을 이동
- 소스의 64 비트를 목적지의 64 비트로 복사

제약 조건

- 두 피연산자가 모두 메모리일 수 없다.
- 즉시값은 사용할 수 없다.

예시

```
movsd xmm0, qword [y]
movsd qword [fltDVar], xmm1
movsd xmm3, xmm2
```

[3] 왜 메모리 ↔ 메모리 이동이 안되는가?

CPU 는 대부분의 명령어에서 한 번에 한 쪽 메모리만 접근을 허용한다.

따라서

```
movsd qword [a], qword [b] ; 불가능
```

대신

```
movsd xmm0, qword [b]
movsd qword [a], xmm0
```

처럼 레지스터를 중간 매개체로 사용해야 한다.

[4] 예제: 변수 복사

다음과 같은 데이터가 있다고 하자.

```
fSVar1 dd 3.14
fSVar2 dd 0.0

fDVar1 dq 6.28
fDVar2 dq 0.0
```

다음 연산을 수행하려 한다.

```
fSVar2 = fSVar1  
fDVar2 = fDVar1
```

단정밀도 복사

```
movss xmm0, dword [fSVar1]  
movss dword [fSVar2], xmm0
```

배정밀도 복사

```
movsd xmm1, qword [fDVar1]  
movsd qword [fDVar2], xmm1
```

레지스터 간 복사

```
movss xmm2, xmm0    ; 32 비트 복사  
movsd xmm3, xmm1    ; 64 비트 복사
```

[5] 타입 지정(dword, qword)을 쓰는가?

어떤 명령어에서는 크기를 생략할 수 있다.

예를 들어, 다른 피연산자가 크기를 이미 명확히 하고 있는 경우이다.

그러나 다음과 같은 이유로 명시하는 것이 좋다.

- 가독성 향상
- 디버깅 용이
- 타입 실수 방지
- 좋은 프로그래밍 습관 유지

따라서 일관성을 위해

- 32 비트: dword
- 64 비트: qword

를 명확히 작성하는 것이 바람직하다.

[18-4] 정수 / 부동소수점 변환 명령어

부동소수점 계산을 수행하는 과정에서 정수 값이 필요한 경우가 자주 발생한다. 그러나 XMM 레지스터 기반 부동소수점 연산에서는 정수와 부동소수점 값을 직접 섞어 연산할 수 있다.

따라서 반드시 명시적인 형 변환(convert)이 필요하다.

또한 하나의 계산 과정에서 단정밀도(32 비트)와 배정밀도(64 비트)가 혼합된다면, 연산은 반드시 일관된 타입으로 수행되어야 한다.

예를 들어,

- float + double : 반드시 둘 중 하나로 변환 후 연산
- int + float : int 를 float 로 변환 후 연산

이러한 이유 때문에 변환 명령어는 부동소수점 연산에서 매우 중요한 역할을 한다.

- 연산 전에 타입을 통일해야 한다.
- 정수 ↔ 부동소수점은 자동 변환되지 않는다.
- 항상 명시적으로 변환해야 한다.

[1] 단정밀도 → 배정밀도 변환

cvtss2sd

cvtss2sd <RXdest>, <RXsrc>

- 32 비트 단정밀도(float)를
- 64 비트 배정밀도(double)로 변환

특징

- 목적지는 반드시 XMM 레지스터
- 소스는 XMM 레지스터 또는 메모리
- 즉시값 불가

예시

cvtss2sd xmm0, dword [fltSVar]

cvtss2sd xmm3, xmm2

이 명령은 정밀도를 확장하는 것이므로 정밀도 손실이 없다.

[2] 배정밀도 → 단정밀도 변환

cvtsd2ss cvttss2ss

cvtsd2ss <RXdest>, <RXsrc>

cvttss2ss <RXdest>, <RXsrc>

- 64 비트 → 32 비트 변환
- 정밀도 손실 발생 가능

여기서 중요한 차이:

명령어	동작
cvtsd2ss	반올림 수행
cvttss2ss	절단(truncate) 수행

예시

```
cvtsd2ss xmm0, qword [fltDVar]  
cvtsd2ss xmm1, xmm5
```

배정밀도 값이 단정밀도로 줄어들기 때문에, 소수점 이하 정밀도가 손실될 수 있다.

[3] 단정밀도 → 정수 변환

cvtss2si cvttss2si

```
cvtss2si <reg>, <RXsrc>  
cvttss2si <reg>, <RXsrc>
```

- 32 비트 float → 32 비트 또는 64 비트 정수

명령어	동작
cvtss2si	반올림
cvttss2si	절단

예시

```
cvtss2si eax, xmm0  
cvtss2si rbx, dword [fltSVar]  
  
cvttss2si eax, xmm0  
cvttss2si rcx, dword [fltSVar]
```

여기서 목적지는 반드시 일반 목적 레지스터(RAX, EAX 등)이다.

[4] 배정밀도 → 정수 변환

cvtsd2si cvttsd2si

```
cvtsd2si <reg>, <RXsrc>  
cvttsd2si <reg>, <RXsrc>
```

- 64 비트 double → 32 비트 또는 64 비트 정수

예시

```
cvtsd2si eax, xmm0  
cvttsd2si rbx, xmm0  
cvttsd2si eax, qword [fltDVar]
```

마찬가지로:

- 목적지는 일반 레지스터
- 소스는 XMM 또는 메모리
- 즉시값 불가

[5] 정수 → 단정밀도 변환

cvtsi2ss

cvtsi2ss <RXdest>, <src>

- 32 비트 또는 64 비트 정수
- → 32 비트 float

예시

cvtsi2ss xmm0, eax
cvtsi2ss xmm0, ebx
cvtsi2ss xmm0, dword [someInt]

목적지는 반드시 XMM 레지스터이다.

[6] 정수 → 배정밀도 변환

cvtsi2sd

cvtsi2sd <RXdest>, <src>

- 32 비트 정수
- → 64 비트 double

예시

cvtsi2sd xmm0, eax
cvtsi2sd xmm0, dword [someInt]

[7] 변환 명령어의 핵심 정리

방향	명령어
float → double	cvtss2sd
double → float	cvtsd2ss
float → int	cvtss2si
double → int	cvtsd2si
int → float	cvtsi2ss
int → double	cvtsi2sd

[18-5] 부동소수점 산술 명령어

부동소수점 산술 명령어는 다음과 같은 연산을 수행한다.

- 덧셈
- 뺄셈
- 곱셈
- 나눗셈

여기서는 기본적인 덧셈과 뺄셈을 설명한다.

[18-5-1] 부동소수점 덧셈

[1] 일반 형식

```
addss <RXdest>, <src>
addsd <RXdest>, <src>
```

동작

```
<RXdest> = <RXdest> + <src>
```

- 결과는 목적지에 저장
- 기존 값은 덮어쓴다

제약 조건

- 목적지는 반드시 XMM 레지스터
- 소스는 즉시값 불가
- 메모리 ↔ 메모리 직접 연산 불가
- 크기는 반드시 동일해야 함(32/64 비트)

[2] 예제

데이터 선언

```
fSNum1 dd 43.75
fSNum2 dd 15.5
fSAns   dd 0.0

fDNum3 dq 200.12
fDNum4 dq 73.2134
fDAns   dq 0.0
```

[2-1] 단정밀도 덧셈

```
movss xmm0, dword [fSNum1]
addss xmm0, dword [fSNum2]
movss dword [fSAns], xmm0
```

[2-2] 배정밀도 덧셈

```
movsd xmm0, qword [fDNum3]
addsd xmm0, qword [fDNum4]
movsd qword [fDAns], xmm0
```

[2-3] 덧셈 명령어 요약

명령어	설명
addss	32비트 float 덧셈
addsd	64비트 double 덧셈

[18-5-2] 부동소수점 뺄셈

[1] 일반 형식

```
subss <RXdest>, <src>
subsd <RXdest>, <src>
```

동작

$\langle \text{RXdest} \rangle = \langle \text{RXdest} \rangle - \langle \text{src} \rangle$

[2] 예제

[2-1] 단정밀도 뺄셈

```
movss xmm0, dword [fSNum1]
subss xmm0, dword [fSNum2]
movss dword [fSAns], xmm0
```

[2-2] 배정밀도 뺄셈

```
movsd xmm0, qword [fDNum3]
subsd xmm0, qword [fDNum4]
movsd qword [fDAns], xmm0
```

[2-3] 뺄셈 명령어 요약

명령어	설명
subss	32비트 float 뺄셈
subsd	64비트 double 뺄셈

[18-5-3] 부동소수점 곱셈

부동소수점 곱셈은 덧셈, 뺄셈과 동일한 구조를 가지며, XMM 레지스터에서 수행된다.
연산 방식은 목적지 레지스터를 누산기(accumulator)처럼 사용하는 구조이다.

[1] 일반 형식

```
mulss <RXdest>, <src>  
mulsd <RXdest>, <src>
```

[2] 연산 의미

```
<RXdest> = <RXdest> * <src>
```

즉,

- 목적지와 소스를 곱한 후
- 결과를 목적지에 덮어쓴다.

목적지는 “입력 + 출력” 역할을 동시에 수행한다.

[3] 제약 조건

1. 목적지는 반드시 XMM 레지스터
2. 소스는 즉시값 불가
3. 소스와 목적지는 같은 크기여야 함 (32 비트 또는 64 비트)
4. 메모리 ↔ 메모리 직접 연산 불가

[4] 왜 메모리 대 메모리 곱셈이 안 되는가?

CPU 는 한 명령어에서 하나의 메모리 피연산자만 허용한다.

따라서

불가능:

```
mulss dword [a], dword [b]
```

가능:

```
movss xmm0, dword [a]  
mulss xmm0, dword [b]
```

레지스터를 중간 매개체로 사용해야 한다.

[5] 예제

데이터 선언

```
fSNum1 dd 43.75
fSNum2 dd 15.5
fSAns   dd 0.0

fDNum3 dq 200.12
fDNum4 dq 73.2134
fDAns   dq 0.0
```

[5-1] 단정밀도 곱셈

```
movss xmm0, dword [fSNum1]
mulss xmm0, dword [fSNum2]
movss dword [fSAns], xmm0
```

[5-2] 배정밀도 곱셈

```
movsd xmm0, qword [fDNum3]
mulsd xmm0, qword [fDNum4]
movsd qword [fDAns], xmm0
```

[5-3] 곱셈 명령어 요약

mulss

- 32 비트 float 곱셈
- 결과는 목적지에 저장

mulsd

- 64 비트 double 곱셈
- 결과는 목적지에 저장

[6] 덧셈/뺄셈/곱셈의 공통 구조

모두 같은 패턴이다.

```
movss xmm0, [a]
opss  xmm0, [b]
movss [result], xmm0
```

이 구조는 부동소수점 산술의 기본 템플릿이다.

[18-5-4] 부동소수점 나눗셈

나눗셈도 동일한 누산기 구조를 사용한다.

[1] 일반 형식

```
divss <RXdest>, <src>  
divsd <RXdest>, <src>
```

[2] 연산 의미

```
<RXdest> = <RXdest> / <src>
```

즉, 목적지 값을 소스로 나눈다.
순서를 반드시 기억해야 한다.

```
divss xmm0, xmm1
```

```
xmm0 = xmm0 / xmm1
```

이지, 반대가 아니다.

[3] 제약 조건

- 목적지는 XMM 레지스터
- 소스는 즉시값 불가
- 크기 일치 필요
- 메모리 ↔ 메모리 불가

[4] 예제

[4-1] 단정밀도 나눗셈

```
movss xmm0, dword [fSNum1]  
divss xmm0, dword [fSNum2]  
movss dword [fSAns], xmm0
```

[4-2] 배정밀도 나눗셈

```
movsd xmm0, qword [fDNum3]  
divsd xmm0, qword [fDNum4]  
movsd qword [fDAns], xmm0
```

[5] 나눗셈에서 주의할 점

1. 0 으로 나누면 예외 또는 NaN 발생 가능
2. 정수 나눗셈과 달리 부동소수점은 오버플로/언더플로가 IEEE 규칙에 따름
3. 결과가 무한대(inf)일 수 있음

부동소수점은 정수 연산보다 예외 상황이 훨씬 많다.

[18-5-5] 부동소수점 제공근

제공근 명령어는 이항 연산이 아니라 단항 연산이다.

[1] 일반 형식

```
sqrtss <RXdest>, <src>
sqrtsd <RXdest>, <src>
```

[2] 연산 의미

$\langle \text{dest} \rangle = \sqrt{\langle \text{src} \rangle}$

소스의 제공근을 계산하여 목적지에 저장한다.

[3] 제약 조건

- 목적지는 XMM 레지스터
- 소스는 즉시값 불가
- 크기 일치 필요
- 메모리 ↔ 메모리 불가

[4] 예제

[4-1] 단정밀도 제공근

```
sqrtss xmm0, dword [fSNum1]
movss dword [fSAns], xmm0
```

[4-2] 배정밀도 제공근

```
sqrtsd xmm0, qword [fDNum3]
movsd qword [fDAns], xmm0
```

[5] 음수에 대한 제공근

음수 값에 대해 sqrt 를 수행하면

- 결과는 NaN
- IEEE-754 규칙 적용

[18-5-6] 전체 구조 정리

연산	32비트	64비트
덧셈	addss	addsd
뺄셈	subss	subsd
곱셈	mulss	mulsd
나눗셈	divss	divsd
제곱근	sqrtss	sqrtsd

[18-6] 부동소수점 제어 명령어

[1] 제어 명령어의 의미

제어 명령어란 프로그램의 실행 흐름을 바꾸는 명령어를 의미한다.

대표적인 구조:

- if
- if-else
- while
- for

정수의 경우 7장에서 배운 `cmp` + 조건부 점프(`je`, `jg` 등)를 사용하여 제어 흐름을 구성한다.
그러나 부동소수점 값은 `cmp`로 비교할 수 없다.

이유는 다음과 같다.

[2] 왜 `cmp`를 사용할 수 없는가?

정수는 이진 정수 표현을 사용한다.

`cmp`는 내부적으로 두 값을 뺀 뒤 결과에 따라 `rFlags`를 설정한다.

하지만 부동소수점은

- IEEE-754 형식
- 부호(sign)
- 지수(exponent)
- 가수(mantissa)

로 구성된다.

정수처럼 단순히 비트 패턴을 비교하면 수학적 크기 비교가 되지 않는다.

따라서 전용 부동소수점 비교 명령어가 필요하다.

[3] 부동소수점 비교 명령어

[3-1] 기본 형식

`ucomiss <xmm_reg>, <src>` ; 32-bit float
`ucomisd <xmm_reg>, <src>` ; 64-bit double

[3-2] 피연산자 규칙

항목	규칙
첫 번째 피연산자	반드시 <code>xmm</code> 레지스터
두 번째 피연산자	<code>xmm</code> 레지스터 또는 메모리
즉시값	사용 불가
결과 저장	<code>rFlags</code>
피연산자 변경	없음

[3-3] ordered vs unordered 비교

부동소수점 비교에는 두 가지 방식이 있다.

Ordered 비교

- NaN 이 포함되면 예외 발생 가능

Unordered 비교(ucomiss, ucomisd)

- S-NaN 에 대해서만 예외 발생
- 일반 NaN 은 예외 없이 처리
- GNU C/C++ 컴파일러가 선호

실무에서는 거의 항상 unordered 비교가 사용된다.

[4] rFlags 설정 방식

ucomisd 는 내부적으로 다음을 판단한다:

- 같음
- 작음
- 큼
- unordered(NaN 포함)

그리고 다음 플래그들을 설정한다.

- ZF(Zero Flag)
- CF(Carry Flag)
- PF(Parity Flag)

개념적으로 다음과 같이 이해하면 된다.

비교 결과	ZF	CF
op1 == op2	1	0
op1 < op2	0	1
op1 > op2	0	0

따라서 우리는 unsigned 조건부 점프를 사용한다.

[5] 사용 가능한 조건부 점프

명령어	의미
je	같다
jne	다르다
jb	<
jbe	≤
ja	>
jae	≥

중요: 부동소수점 비교는 signed 개념이지만 조건 판단은 unsigned 점프를 사용한다.

[6] IF 문 구현 원리

어셈블리는 고급 구조를 직접 제공하지 않는다.

예:

```
if (a > b)
    doSomething();
```

구조적으로는 다음과 같이 바뀐다.

```
비교
조건이 false 이면 점프
true 블록 실행
```

[7] 예제 1: 최대값 갱신

C 코드

```
if (fltNum > fltMax)
    fltMax = fltNum;
```

데이터 선언

```
section .data
fltNum dq 7.5
fltMax dq 5.25
```

어셈블리 코드

```
movsd xmm1, qword [fltNum]
ucomisd xmm1, qword [fltMax]
jbe skipUpdate          ; fltNum <= fltMax 이면 건너뛰

movsd qword [fltMax], xmm1

skipUpdate:
```

동작 분석

1. movsd → fltNum 을 xmm1 에 로드
2. ucomisd → fltNum ? fltMax
3. jbe → 작거나 같으면 갱신하지 않음
4. 크면 저장

논리 구조는 다음과 같다.

```
if NOT (fltNum > fltMax)
    jump
```

즉, 항상 조건의 부정을 점프에 사용한다.

[8] IF-ELSE 구현

C 코드

```
if (x != 0.0) {
    ans = x / y;
    errFlg = FALSE;
} else {
    errFlg = TRUE;
}
```

어셈블리 구현

```
movsd xmm1, qword [x]
ucomisd xmm1, qword [fltZero]
je doElse          ; x == 0 이면 else

; ----- then -----
divsd xmm1, qword [y]
movsd qword [ans], xmm1
mov byte [errFlg], FALSE
jmp endIf

; ----- else -----
doElse:
mov byte [errFlg], TRUE

endIf:
```

구조 분석

IF-ELSE 는 반드시 다음 구조를 가진다.

```
비교
조건이 false 면 else 로 점프
then 블록
무조건 점프
else 블록
```

어셈블리에서는 흐름을 직접 설계해야 한다.

[9] 부동소수점 비교의 위험성

부동소수점은 정확하지 않다.

예:

```
0.1 + 0.1 + ... (10 번)
```

수학적으로 1.0

하지만 실제 결과는:

```
0.999999999999...
```

따라서 다음과 같은 코드는 위험하다

```
ucomisd xmm0, qword [one]
je equalLabel
```

[10] 안전한 비교 방법(Epsilon 비교)

실무에서는 다음과 같이 한다.

```
if (fabs(a - b) < epsilon)
```

어셈블리 예:

```
subsd xmm0, xmm1      ; xmm0 = a - b
; 절댓값 처리 필요
ucomisd xmm0, qword [epsilon]
jnb equalLabel
```

이 방식은 오차 허용 범위를 둔다.

[11] 핵심 정리 및 이해

- cmp 는 부동소수점에 사용 불가
- ucomiss/uncomisd 사용
- 결과는 rFlags 에 저장
- unsigned 조건부 점프 사용
- IF-ELSE 구조는 직접 구성
- 실수 비교는 오차 고려 필수

1. xmm 레지스터 로드
2. ucomis* 수행
3. rFlags 설정
4. 조건부 점프
5. 흐름 제어

[18-7] 부동소수점 호출 규약

[1] 기본 원칙

12장에서 설명한 일반 함수 호출 규약은 그대로 적용된다.

즉,

- 인자 전달
- 반환 값
- 스택 사용
- 레지스터 보존 규칙

모두 동일하다.

다만, 부동소수점 레지스터 사용 시 특별히 주의할 점이 있다.

[2] 가장 중요한 규칙

부동소수점 레지스터는 호출 후 보존되지 않는다.

즉,

xmm0 ~ xmm15 는 모두 caller-saved

함수를 호출하면 이 레지스터들의 값이 유지된다는 보장이 없다.

[3] 인자 전달 방식

첫 8 개의 부동소수점 인자

xmm0 ~ xmm7

에 순서대로 전달된다.

예:

double f(double a, double b, double c);

호출 시:

xmm0 → a

xmm1 → b

xmm2 → c

[4] 9 번째 이후 인자

8 개를 초과하면

- 스택에 push
- 역순으로 저장

이는 12 장에서 설명한 일반 호출 규약과 동일한다.

[5] 반환값

부동소수점 함수의 반환값은

xmm0

에 저장된다.

[6] 왜 주의해야 하는가?

예를 들어,

movsd, xmm1, [value]

call someFunc

someFunc 가 xmm1 을 덮어쓸 수 있다.

따라서 필요하다면

- 스택에 저장하거나
- 다른 보존 레지스터로 이동

해야 한다.

[18-8] 예제: Sum and Average

이 프로그램은

1. 부동소수점 배열의 합을 구함
2. 평균을 계산함

```
; Floating-Point Example Program
; *****
section .data
; -----
; Define constants.
NULL equ 0 ; end of string
TRUE equ 1
FALSE equ 0
EXIT_SUCCESS equ 0 ; Successful operation
SYS_exit equ 60 ; system call code for terminate
; -----
```

```

fltLst dq 21.34, 6.15, 9.12, 10.05, 7.75
dq 1.44, 14.50, 3.32, 75.71, 11.87
dq 17.23, 18.25, 13.65, 24.24, 8.88
length dd 15
lstSum dq 0.0
lstAve dq 0.0
; *****
section .text
global _start
_start:
; -----
; Loop to find floating-point sum.
mov ecx, [length]
mov rbx, fltLst
mov rsi, 0
movsd xmm1, qword [lstSum]
sumLp:
movsd xmm0, qword [rbx+rsi*8] ; get fltLst[i]
addsd xmm1, xmm0 ; update sum
inc rsi ; i++
loop sumLp
movsd qword [lstSum], xmm1 ; save sum
; -----
; Compute average of entire list.
cvtsi2sd xmm0, dword [length]
cvtsd2si dword [length], xmm0
divsd xmm1, xmm0
movsd qword [lstAve], xmm1
; -----
; Done, terminate program.
last:
mov rax, SYS_exit
mov rbx, EXIT_SUCCESS
syscall ; exit w/success

```

[1] 데이터 선언

```
fltLst dq 21.34, 6.15, 9.12, ...  
length dd 15  
lstSum dq 0.0  
lstAve dq 0.0
```

- dq → double (8 바이트)
- length → 정수
- 합과 평균 → double

[2] 합 구하기 루프

코드

```
mov ecx, [length]  
mov rbx, fltLst  
mov rsi, 0  
movsd xmm1, qword [lstSum]
```

레지스터 역할

레지스터	용도
ecx	루프 카운터
rbx	배열 시작 주소
rsi	인덱스
xmm1	누적 합

루프 본체

```
sumLp:  
movsd xmm0, qword [rbx+rsi*8]  
addsd xmm1, xmm0  
inc rsi  
loop sumLp
```

[2-1] 동작 분석

[2-1-1] 주소 계산

```
rbx + rsi * 8
```

double 은 8 바이트 이므로:

```
배열[i] = base + i * 8
```

[2-1-2] 누적

```
xmm1 = xmm1 + xmm0
```

[2-1-3] loop 명령어

```
loop sumLp
```

내부적으로

```
ecx--
```

```
if ecx != 0 → 점프
```

loop 명령어는 ECX 를 자동 사용한다.

[2-1-3] 합 저장

```
movsd qword [lstSum], xmm1
```

[2-2] 평균 계산

```
cvtsi2sd xmm0, dword [length]
```

```
divsd xmm1, xmm0
```

```
movsd qword [lstAve], xmm1
```

[2-2-1] 정수 → double 변환

```
cvtsi2sd
```

필요한 이유

length 는 정수

sum 은 double

float 연산을 하려면 같은 타입이어야 한다.

[2-2-2] 나눗셈

```
xmm1 = xmm1 / xmm0
```

책 코드의 의심스러운 줄

```
cvtsd2si dword [length], xmm0
```

이 줄은 의미상 불필요하거나 오타 가능성이 있다.

- double → int 변환

- length 를 덮어씀

평균 계산에는 필요하지 않다.

[2-3] 프로그램 종료

```
mov rax, SYS_exit
mov rbx, EXIT_SUCCESS
syscall
```

- rax: 시스템 콜 번호
- rbx: 종료 코드

[18-8] 예제: Absolute Value

목표:

```
|x|
```

```
; Floating-Point Absolute Value Example
section .data
; -----
; Define constants.
TRUE equ 1
FALSE equ 0
SUCCESS equ 0 ; successful operation
SYS_exit equ 60 ; call code for terminate
; -----
; Define some test variables.
dZero dq 0.0
dNegOne dq -1.0
fltVal dq -8.25
; *****

section .text
global _start
_start:
; -----
; Perform absolute value function on fltVal
movsd xmm0, qword [fltVal]
ucomisd xmm0, qword [dZero]
jae isPos

mulsd xmm0, qword [dNegOne]
movsd qword [fltVal], xmm0
isPos:
; -----
; Done, terminate program.
last:
```

```
mov rax, SYS_exit
mov rbx, EXIT_SUCCESS
syscall ; exit w/success
```

[1] 데이터

```
dZero    dq 0.0
dNegOne  dq -1.0
fltVal   dq -8.25
```

[2] 코드

```
movsd xmm0, qword [fltVal]
ucomisd xmm0, qword [dZero]
jae isPos
mulsd xmm0, qword [dNegOne]
movsd qword [fltVal], xmm0
isPos:
```

[3] 동작 원리

[3-1] 비교

```
xmm0 >= 0 ?
```

[3-2] jae

```
jump if above or equal
```

즉,

```
if xmm0 >= 0 → 점프
```

[3-3] 음수일 경우

```
xmm0 = xmm0 * (-1.0)
```

곱셈으로 부호 반전

[4] 구현한 이유

부동소수점은

- 정수처럼 neg 명령어 없음
- sign 비트 직접 조작도 가능하지만
- 교육용으로는 곱셈이 명확함

[5] 더 나은 절댓값 구현 방법(고급)

사실 SSE 에는:

andpd

같은 비트 마스킹 방법으로 sign bit 를 제거할 수 있다.

이 방식이 더 빠르다.

Chapter19. 병렬 처리(Parallel Processing)

[1] 동시성과 병렬성의 정확한 의미

컴퓨팅에서 동시성(concurrency)은 여러 작업이 동시에 진행되는 것처럼 보이는 실행 모델을 의미한다.

여기서 핵심은 “동시에 보인다”는 점이다.

동시성은 두 가지 방식으로 구현될 수 있다.

[1-1] 실제 병렬 실행(True Parallelism)

- 프로세스 A → 코어 0
- 프로세스 B → 코어 1
- 물리적으로 동시에 실행됨

이것이 진짜 병렬 처리(parallel processing)이다.

[1-2] 단일 코어에서의 교차 실행(Interleaving)

- A 실행 → 일시 중지
- B 실행 → 일시 중지
- 다시 A 실행
- 빠르게 반복

운영체제의 스케줄러가 CPU 시간을 잘게 나누어 배분하기 때문에, 사용자는 동시에 실행되는 것처럼 느끼게 된다.

개념	의미
동시성	동시에 보이도록 만드는 실행 구조
병렬성	물리적으로 동시에 실행되는 것

[2] 병렬 처리의 근본 아이디어

병렬 처리의 기본 전제는 다음과 같다.

- 큰 문제를 여러 개의 독립적인 하위 문제로 분해할 수 있다.
- 그리고 그 하위 문제들이 서로 간섭 없이 동시에 실행될 수 있다.

그렇다면 전체 실행 시간은

직렬 실행 시간 ÷ 사용 가능한 실행 단위 수
에 가까워질 수 있다.

하지만 여기에는 중요한 조건이 있다.

- 하위 문제는 서로 독립적이어야 한다.
- 중간 결과 결합 비용이 작아야 한다.
- 동기화 비용이 병렬화 이득보다 작아야 한다.

모든 문제가 병렬화 가능한 것은 아니다.

[19-1] 분산 컴퓨팅(Distributed Computing)

[1] 개념 구조

분산 컴퓨팅은

- 여러 대의 물리적으로 다른 컴퓨터가 네트워크를 통해 협력하는 구조

기본 구조

Master Node



Compute Nodes (1...N)



결과 수집 → 최종 결합

마스터가 작업을 분배하고, 각 계산 노드는 부분 계산을 수행한 후 결과를 반환한다.

[2] 대표 사례

Folding@home

- 단백질 접힘 시뮬레이션
 - 계산 약물 설계
 - 분자 동역학 계산
 - 100,000 대 이상의 자원봉사 PC 사용
- 이 방식은 수십만 대까지 확장 가능하다.

[3] 장점과 단점

장점

- 거의 무제한 확장 가능
- 지리적으로 분산 가능
- 고가용성 설계 가능

단점

- 네트워크 지연
- 통신 오버헤드
- 장애 복구 복잡성

[19-2] 다중 처리(Multiprocessing)

[1] 개념

하나의 CPU 칩에는 여러 개의 코어가 있다.
모든 코어는 동일한 메모리에 접근할 수 있다.

이 환경에서:

하나의 프로세스가 여러 개의 스레드를 생성하여 동시에 실행
이것이 공유 메모리 병렬 처리(shared-memory parallelism)이다.

[2] 스레드의 특징

스레드는:

- 초기 프로세스의 주소 공간 공유
- 동일한 메모리 접근
- 생성/종료 속도가 빠름
- 커널 자원 재할당 없음

그래서 light-weight process 라고 부름

[3] 분산 방식과 비교

항목	분산	스레드
확장성	매우 큼	코어 수에 제한
통신	네트워크	공유 메모리
지연	높음	낮음
동기화	메시지 기반	메모리 기반

[19-2-1] POSIX Threads(pThreads)

[1] pThreads 란?

Unix/Linux 계열에서 사용하는 표준 스레드 라이브러리

주요 API

```
pthread_create()
pthread_join()
```

[2] 스레드 생성

```
mov rdi, pthreadID0
mov rsi, NULL
mov rdx, threadFunction0
mov rcx, NULL
call pthread_create
```

인자 의미

레지스터	의미
rdi	스레드 ID 저장 위치
rsi	속성
rdx	함수 주소
rcx	인자

[3] 스레드 종료 대기

```
mov rdi, qword [pthreadID0]
mov rsi, NULL
call pthread_join
```

join 은 blocking call 이다.

[4] C 예제

```
#include <pthread.h>

void* threadFunc(void* arg) {
    for (int i = 0; i < MAX/2; i++)
        myValue = (myValue / X) + Y;
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, threadFunc, NULL);
    pthread_create(&t2, NULL, threadFunc, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}
```

[19-2-2] 레이스 조건(Race Conditions)

[1] 정의

두 개 이상의 스레드가 동시에 같은 메모리 위치에 쓰기를 수행하는 상황

특히:

- 읽기 → 계산 → 쓰기
- 이 세 단계가 원자적(atomic)이지 않기 때문에 발생

[2] 문제 예제

공식:

$$\text{myValue} = (\text{myValue} / X) + Y$$

단순화:

$$X = 1$$
$$Y = 1$$

결과는

$$\text{myValue}++$$

MAX 번 반복하면 결과는 MAX 여야 한다.

[3] 어셈블리 수준에서 문제 발생

문제는 다음 3 단계 때문이다.

1. 메모리에서 myValue 읽기
2. 계산 수행
3. 다시 메모리에 저장

이 3 단계는 하나의 명령어가 아니다.

[4] 충돌 시나리오

현재 myValue = 730

코어 0

```
mov rax, [myValue] → 730  
add rax, 1 → 731
```

코어 1

```
mov rax, [myValue] → 730  
add rax, 1 → 731
```


그 후:

코어 0 → 731 저장

코어 1 → 731 저장

결과: 731

- 실제 기대값: 732
- 한 번 증가가 손실됨

[5] 왜 이게 무서운가?

- 실행마다 결과가 다름
- 디버깅 어려움
- MAX 가 작으면 문제 안 보임
- MAX 가 커지면 터짐

이런 종류의 버그는: “가끔 발생하는 이상한 값” 형태로 나타난다.

[6] 근본 원인

CPU 는 다음을 보장하지 않는다.

read → modify → write

가 원자적으로 수행된다고

각 코어는:

- 독립적인 레지스터
- 독립적인 실행 파이프라인
- 캐시 계층 존재

따라서 메모리 일관성 문제까지 포함하면 상황은 더 복잡해진다.

[7] 실제 스레드 함수의 어셈블리 예

threadFunc:

```
mov ecx, MAX/2
```

loopStart:

```
mov rax, [myValue]
```

```
add rax, 1
```

```
mov [myValue], rax
```

```
loop loopStart
```

```
ret
```

이 함수가 두 코어에서 동시에 실행되면

- 읽기 시점이 겹치면 손실 발생
- 겹침 정도는 예측 불가
- 실행할 때마다 결과 달라질 수 있음

[8] 해결 방법

레이스 조건을 해결하려면

- mutex
- atomic instruction
- compare-and-swap
- spinlock
- semaphore

등이 필요하다.

예:

```
lock inc qword [myValue]
```

은 원자적 증가를 보장한다.

[9] 핵심 정리

이 장의 가장 중요한 메시지는 이것이다.

“병렬 처리는 빠르지만, 공유 메모리는 위험하다.”

그리고 레이스 조건은 고급 언어 문제가 아니라 CPU 아키텍처 수준의 문제다.

최종 구조 요약

병렬 처리

└─ 분산 컴퓨팅 (네트워크 기반)

└─ 다중 처리 (공유 메모리 기반)

└─ 스레드

└─ 레이스 조건 문제

Chapter20. 인터럽트(Interrupts)

일반적인 의미에서 인터럽트는 현재 진행 중인 작업을 일시적으로 중단하고 다른 작업을 처리한 뒤, 다시 원래 작업으로 복귀하는 것을 의미한다.

예를 들어, 전화 통화를 하고 있는 도중 초인종이 울리면 통화를 잠시 멈추고 방문자를 응대한 뒤, 다시 통화를 이어가는 상황과 유사하다.

컴퓨터 시스템에서도 인터럽트는 동일한 개념으로 작동한다.

현재 실행 중인 프로세스를 일시 중단하고, 다른 작업을 수행한 후, 이후 다시 원래 실행 지점으로 복귀한다.

보다 기술적으로 정의하면,

- 인터럽트는 CPU가 실행하는 명령어의 흐름을 변경시키는 이벤트이다.

이러한 이벤트는 소프트웨어 또는 하드웨어에 의해 생성될 수 있다.

대표적인 예:

- I/O 장치가 데이터 전송 완료를 알리는 경우
- 프로그램이 운영체제 서비스 요청을 수행하는 경우
- 예기치 않은 오류 상황이 발생한 경우

인터럽트 처리는 매우 민감한 작업이다.

인터럽트는 언제든지 발생할 수 있으며, 하나의 인터럽트가 또 다른 인터럽트에 의해 중단될 수 있다.

[20-1] 다중 처리(Multiprocessing)

현대 운영체제는 여러 프로그램이 동시에 실행되거나, 동시에 실행되는 것처럼 보이도록 지원한다.

이를 위해 운영체제는 다음과 같은 자원을 관리한다.

- CPU 코어
- 주기억장치(RAM)
- 보조 저장장치(SSD/HDD)
- 디스플레이
- 키보드 및 마우스

이 자원들은 여러 프로그램이 공유해야 한다.

이때 인터럽트 메커니즘은 자원 공유를 가능하게 하는 핵심 기술이다.

인터럽트가 발생하면:

1. 현재 프로세스가 중단되다.
2. 인터럽트가 처리된다.
3. 이후 원래 프로세스가 재개된다.
4. 필요하다면 다른 프로세스로 전환될 수도 있다.

인터럽트는 인터럽트 서비스 루틴(ISR, Interrupt Service Routine)이라는 특별한 소프트웨어 코드에 의해 처리된다.

ISR 은 인터럽트 핸들러 또는 디바이스 드라이버라고도 불린다.

이러한 빠른 전환 덕분에 모든 프로세스가 동시에 실행되는 것처럼 보이게 된다.

단, 모든 코드가 인터럽트 가능한 것은 아니다.

운영체제 내부의 일부 중요한 커널 영역은 절대로 인터럽트되어서는 안된다.

[20-1-1] 인터럽트 분류

인터럽트를 이해하기 위해서는 두 가지 관점이 중요하다.

1. 발생 시점(Timing)
2. 발생 주체(Hardware vs Software)

[20-1-2] 인터럽트 발생 시점(Interrupt Timing)

인터럽트는 발생 시점에 따라 다음과 같이 구분된다.

[1] 비동기 인터럽트(Asynchronous Interrupts)

프로그램 실행 흐름과 무관하게 임의의 시점에 발생한다.

특징:

- 예측 불가능
- 현재 실행 위치와 관계 없음
- 외부 하드웨어에 의해 주로 발생

예:

- 키보드 입력
- 네트워크 패킷 수신
- 타이머 인터럽트

[2] 동기 인터럽트(Synchronous Interrupts)

현재 실행 중인 명령어와 직접적인 관련이 있다.

특징:

- 특정 위치에서 발생
- 동일 조건에서 반복 발생 가능
- CPU 제어 하에서 발생

예:

- 0 으로 나누기
- 잘못된 메모리 접근
- 시스템 콜 호출

[20-1-3] 인터럽트 분류

[1] 하드웨어 인터럽트

하드웨어 장치에 의해 발생한다.

예:

- I/O 장치
- 인터벌 타이머
- 다중 CPU 시스템의 다른 프로세서

하드웨어 인터럽트는 비동기적이다.

키보드 예시

키가 눌리면

1. 키보드가 인터럽트 신호 발생
2. CPU가 현재 작업 중단
3. ISR 실행
4. 키 값 저장
5. 원래 작업 복귀

이 과정은 매우 빠르게 수행된다.

[1-1] 예외

예외는 현재 프로세스에 의해 발생하는 동기적 인터럽트이다.

종류:

Fault:

- 예: 페이지 폴트
- 처리 후 원래 위치에서 재시작

Trap

- 주로 디버깅용
- 연속성 유지

Abort

- 심각한 오류
- 예:
 - 0으로 나누기
 - 불법 명령어
 - 유효하지 않은 메모리 접근
- 대부분 프로세스 종료

[2] 소프트웨어 인터럽트

CPU 명령어 실행 중 발생한다.

- 프로그래머가 명시적으로 요청
- 시스템 콜 수행 시 사용
- 동기적 발생

예:

int 0x80

[20-2] 인터럽트 유형과 수준

[20-2-1] 인터럽트 유형

Maskable Interrupt

- 무시 가능
- 주로 I/O 인터럽트

Non-Maskable Interrupt(NMI)

- 무시 불가
- 즉시 처리
- 치명적인 하드웨어 오류

[20-2-2] 특권 수준(Privilege Levels)

CPU 는 네 가지 권한 수준을 가진다.

Level	설명
0	커널 모드 (모든 하드웨어 접근 가능)
1	제한적 커널 서비스
2	더 제한적 접근
3	사용자 모드 (응용 프로그램)

응용 프로그램은 Level 3 에서 실행된다.

키보드 인터럽트가 발생하면 ISR 은 Level 0 에서 실행되어야 한다.
인터럽트 메커니즘은 이러한 특권 상승과 복원을 안전하게 수행한다.

[20-3] 인터럽트 처리 과정

인터럽트가 발생하면 다음 단계가 수행된다.

[20-3-1] 인터럽트 서비스 루틴(ISR)

인터럽트 발생 시 실행되는 코드이다.

ISR 개발은 어려운 이유:

- 동시성 문제
- 레이스 조건
- 디버깅 어려움

[20-3-2] 처리 단계

[1] 중단(Suspension)

현재 실행 중인 프로그램이 중단된다.

CPU 는 최소한 다음을 저장한다.

- rip(다음 실행 주소)
- rFlags(상태 플래그)

예시 개념

```
push rFlags  
push rip
```

[2] ISR 주소 획득

ISR 주소는 IDT(Interrupt Descriptor Table)에 저장되어 있다.

특징:

- 각 항목은 16 바이트
- 최대 256 개 항목
- IDTR 레지스터가 IDT 시작 주소 보관
- Level 0 에서만 접근 가능

ISR 주소 계산

ISR 주소 = IDT 시작 주소 + (인터럽트 번호 × 16)

[3] ISR 로 점프

검증 수행

- 인터럽트 합법성 확인
- 특권 변경 확인

이후:

```
mov rip, ISR_address
```

[4] ISR 실행

필요 시 전체 문맥 저장:

```
push rax  
push rbx  
...
```

Linux 에서는 ISR 이 두 부분으로 나뉜다.

Top-half

- 즉시 실행
- 인터럽트 승인
- 긴급 작업 처리

Bottom-half

- 후속 처리
- 스케줄링 가능
- 비긴급 작업 수행

[5] 재개(Resumption)

문맥 복원

```
pop rax  
pop rbx  
...  
iret
```

iret 명령은 다음을 복원한다.

- rip
- rFlags

원래 프로세스가 다시 실행된다.

[20-4] 인터럽트 처리 요약

처리 순서 요약:

1. 현재 프로그램 중단
2. rip, rFlags 저장
3. IDT 에서 ISR 주소 계산
4. ISR 로 점프
5. ISR 실행
6. 문맥 복원
7. iret 실행
8. 원래 프로세스 재개